
```
clc;
clear all;
close all;
```

Résolution d'un problème inverse: déconvolution d'un signal triangulaire

1. Construction du jeu de données a. Simulation du signal d'entrée (Question 1)

```
% Paramètres de simulation
L = 100; % Durée d'observation en secondes
fe = 1; % Fréquence d'échantillonnage en Hz
Te = 1 / fe; % Période d'échantillonnage en secondes
Nx = L / Te; % Nombre d'échantillons
nx = 0:Nx-1; % Indices temporels
tx = nx * Te; % Base de temps (en secondes)

% Signal triangulaire
x1 = (tx(1:Nx/2)) / (L/2); % Première partie  $\forall t \in [0, L/2]$ 
x2 = (-tx(Nx/2+1:end) + L) / (L/2); % Deuxième partie  $\forall t \in [L/2, L]$ 
x = [x1, x2]; % Signal triangulaire complet

% Affichage du signal d'entrée
figure;
plot(tx, x);
xlabel('Temps (s)');
ylabel('x(t)');
title('Signal triangulaire x(t)');
grid on;
% Interprétation : Le signal triangulaire est bien symétrique et atteint un
maximum en L/2.
```

% b. Simulation du filtre gaussien (Question 2)

```
% Paramètres du filtre gaussien
m = 15; % Moyenne
sigma = 4; % Écart-type
Nh = 30; % Horizon temporel
th = (0:Nh-1)*Te; % Base de temps du filtre

% Réponse impulsionnelle
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((th - m).^2) / (2 * sigma^2));

% Affichage de la réponse impulsionnelle
figure;
plot(th, h, 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('h(t)');
title('Réponse impulsionnelle du filtre gaussien h(t)');
grid on;
% Interprétation : La réponse impulsionnelle a une forme en cloche, typique
```

d'un filtre gaussien.

```
% c. Simulation du problème direct: première approche (Question 5-7)

% Produit de convolution discret
ynb = conv(x, h, 'full'); % Convolution complète
tnb = (0:length(ynb)-1) * Te; % Base de temps associée
% Quantification du signal
y = round(ynb * 2^10) / 2^10;
% Calcul du bruit de quantification
b = y - ynb;
figure;

% Signal de sortie non bruité
subplot(3, 1, 1);
plot(tnb, ynb, 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('ynb(t)');
title('Signal de sortie non bruité y_nb[n]');
grid on;

% Signal de sortie bruité
subplot(3, 1, 2);
stairs(tnb, y, 'LineWidth', 1.5); % stairs pour un affichage "discret"
xlabel('Temps (s)');
ylabel('y(t)');
title('Signal de sortie bruité y[n]');
grid on;

% Bruit de quantification
subplot(3, 1, 3);
plot(tnb, b, 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('b(t)');
title('Bruit de quantification b[n]');
grid on;
% Interprétation : On observe que la quantification introduit un bruit
discret qui perturbe légèrement y.

% d. Simulation du problème direct par approche matricielle (Question 8-10)
Nx = length(x); % Nombre d'échantillons du signal d'entrée x[n]
Nh = length(h); % Nombre d'échantillons de la réponse impulsionnelle h[n]

Ny = Nx + Nh - 1; % Taille complète de la sortie ynb

% Initialisation de la première colonne et de la première ligne de la
matrice Toeplitz
colonne_H = [h(:); zeros(Nx - 1, 1)]; % Première colonne : h suivi de zéros
ligne_H = [h(1), zeros(1, Nx - 1)]; % Première ligne : h(1) suivi de zéros

% Construction de la matrice Toeplitz pour la convolution complète
H = toeplitz(colonne_H, ligne_H);
```

```

% Calcul de la sortie non bruitée par multiplication matricielle
ynb_mat = H * x(:); % Produit matriciel H * x

% Générer la base de temps pour la sortie complète
tnb_full = (0:Ny-1) * Te;

% Affichage du signal obtenu par l'approche matricielle
figure;
plot(tnb_full, ynb, 'r', 'LineWidth', 1.5); hold on;
plot(tnb_full, ynb_mat, 'b--', 'LineWidth', 1.5);
legend('Convolution classique (conv)', 'Multiplication matricielle (H*x)');
xlabel('Temps (s)');
ylabel('Amplitude');
title('Comparaison entre la convolution et l\'approche matricielle sans
troncature');
grid on;
% Interprétation : Les courbes sont superposées, prouvant que l'approche
matricielle donne un résultat identique à la convolution.

% Vérification de la différence entre les deux approches
erreur_convolution = norm(ynb - ynb_mat);
disp(['Erreur entre convolution et approche matricielle : ',
num2str(erreur_convolution)]);

% 2. Déconvolution
% a. Déconvolution « naïve » et constat de l'instabilité (Question 11-13)
% Déconvolution naïve sans bruit
x_rec_naive = deconv(ynb, h);

% Générer la base de temps pour x_rec_naive
t_rec = (0:length(x_rec_naive)-1) * Te;

% Affichage du signal reconstruit sans bruit
figure;
plot(t_rec, x_rec_naive, 'r', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal reconstruit par déconvolution naïve (sans bruit)');
grid on;

% Déconvolution avec le signal bruité y
x_rec_bruite = deconv(y, h);

% Générer la base de temps pour x_rec_bruite
t_rec_bruite = (0:length(x_rec_bruite)-1) * Te;

% Affichage du signal reconstruit avec bruit
figure;
plot(t_rec_bruite, x_rec_bruite, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal reconstruit par déconvolution naïve (avec bruit)');
grid on;
% Interprétation : La déconvolution naïve amplifie fortement le bruit.

```

```
% Tester plusieurs valeurs de sigma
sigma_values = [2, 4, 8, 16]; % Différentes largeurs du filtre gaussien
figure;
hold on;

for sigma_test = sigma_values
    % Calculer la nouvelle réponse impulsionnelle avec le sigma modifié
    h_test = (1 / (sigma_test * sqrt(2 * pi))) * exp(-((th - m).^2) / (2 *
sigma_test^2));

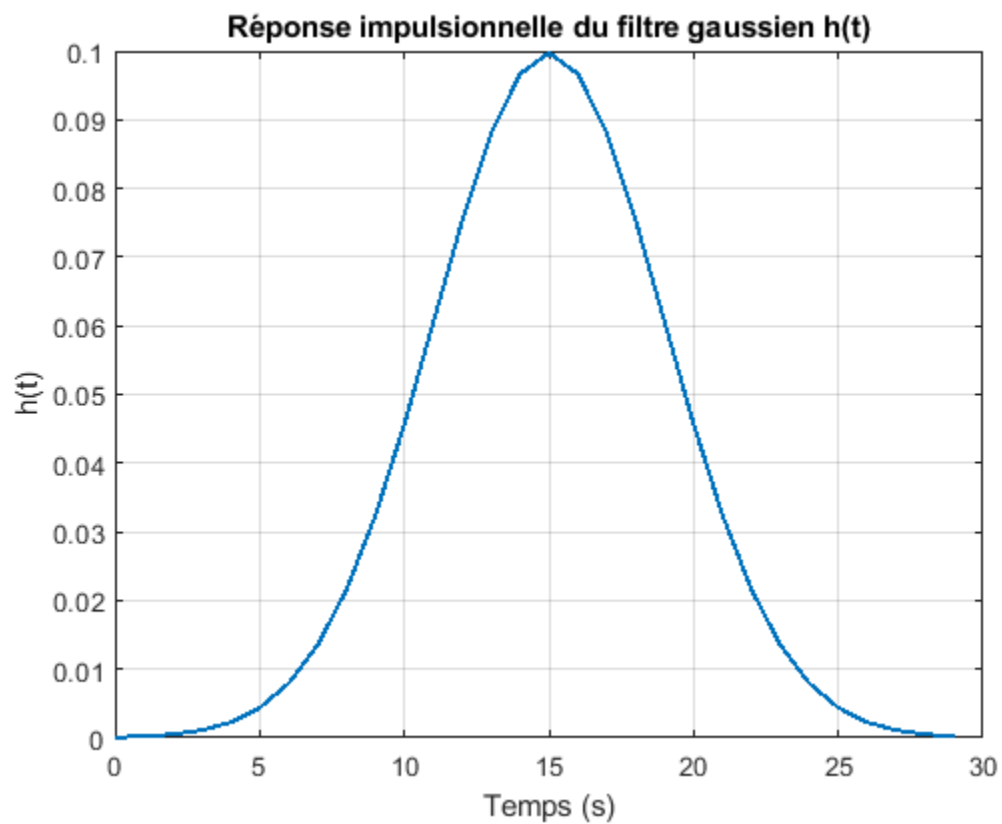
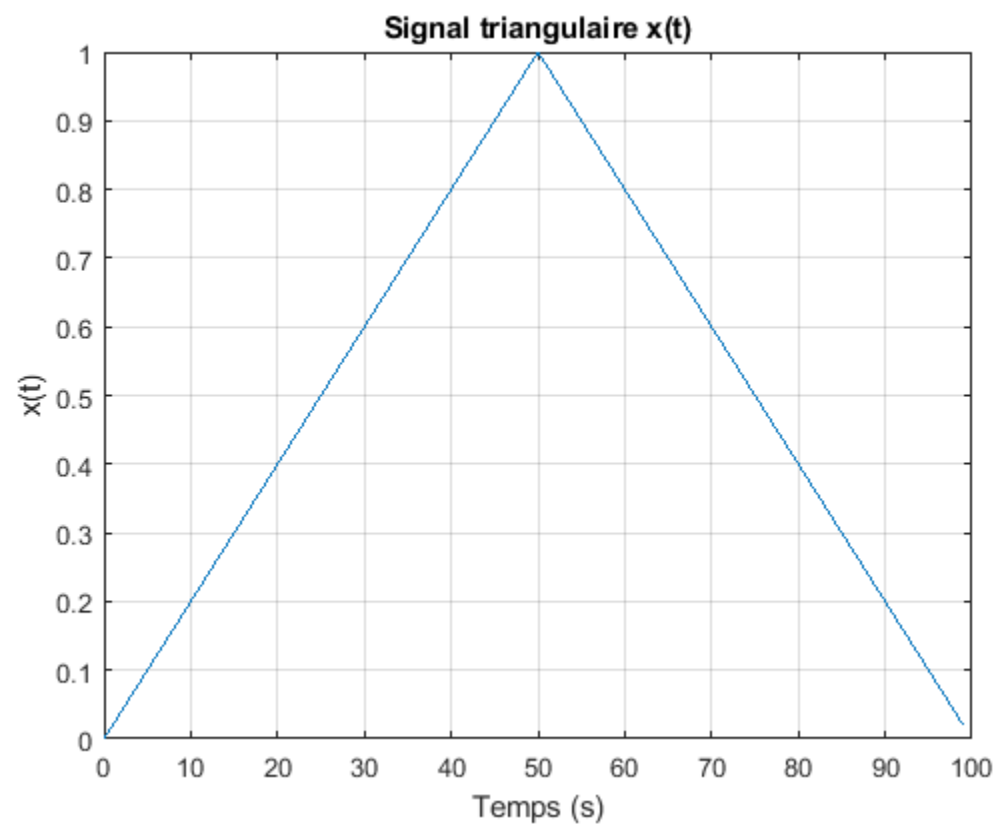
    % Déconvolution avec le nouveau filtre
    x_rec_test = deconv(y, h_test);

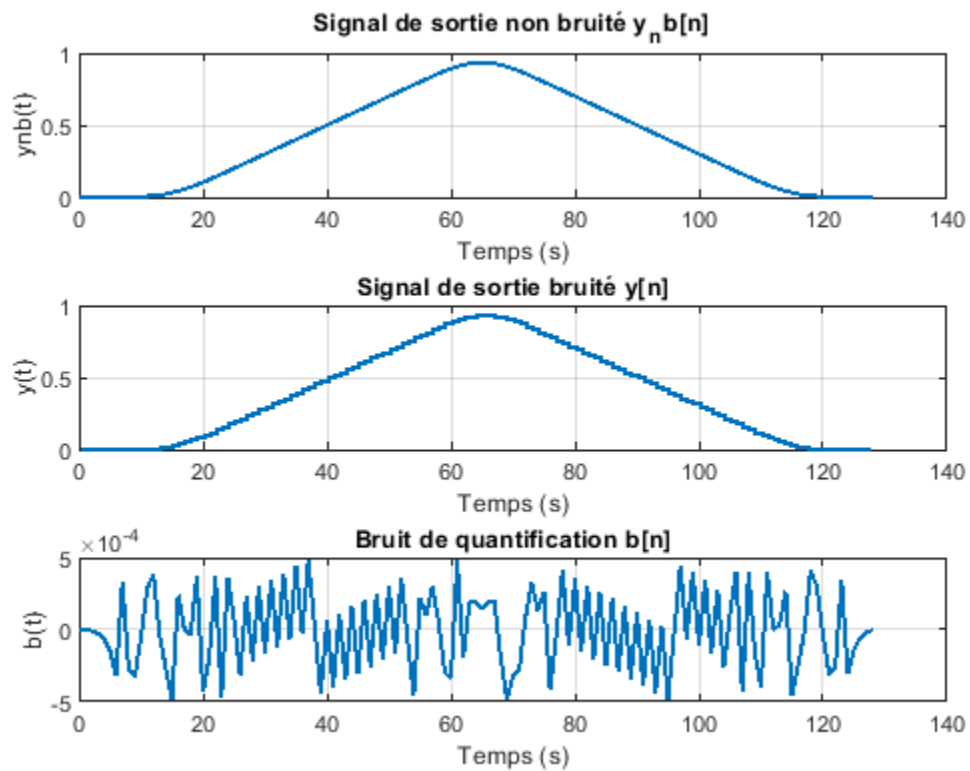
    % Générer la base de temps pour l'affichage
    t_rec_test = (0:length(x_rec_test)-1) * Te;

    % Tracer le signal reconstruit
    plot(t_rec_test, x_rec_test, 'LineWidth', 1.5, 'DisplayName', ['\sigma =
', num2str(sigma_test)]);
end

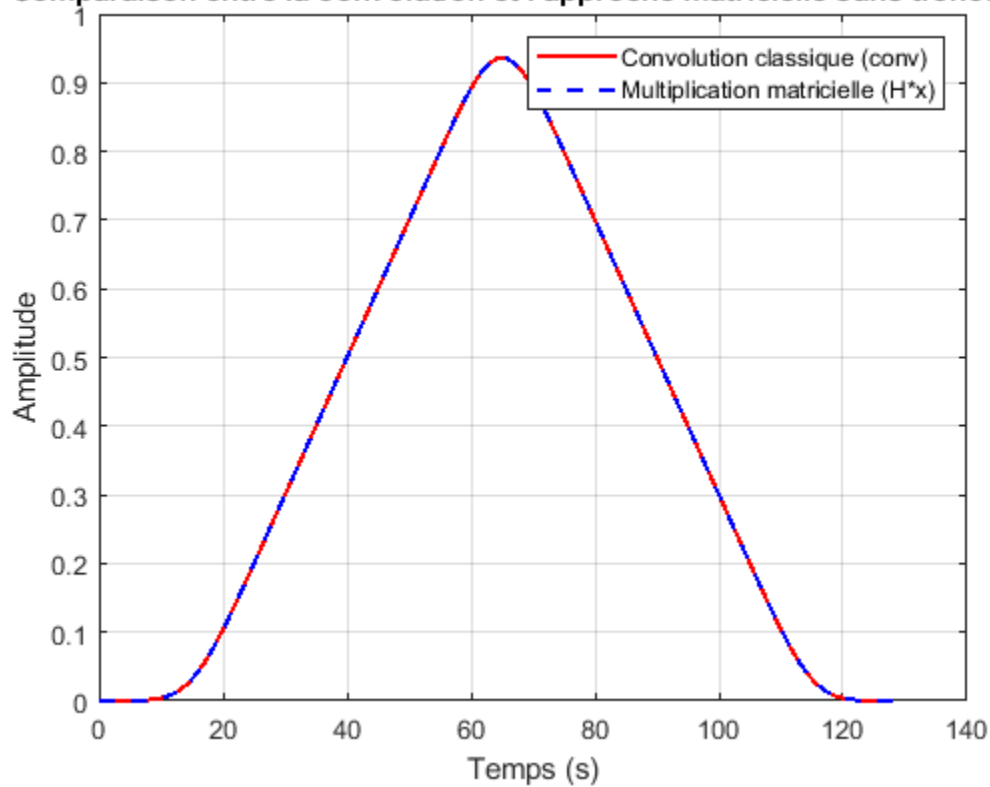
legend();
xlabel('Temps (s)');
ylabel('Amplitude');
title('Effet du paramètre \sigma sur la déconvolution naïve');
grid on;
hold off;

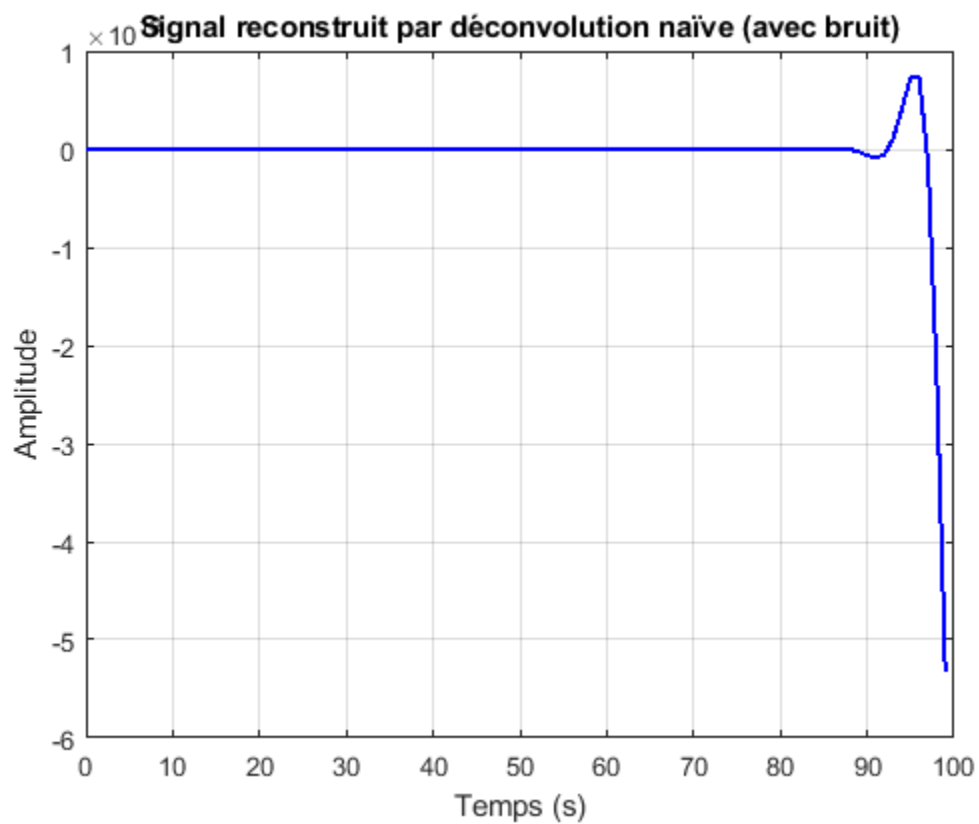
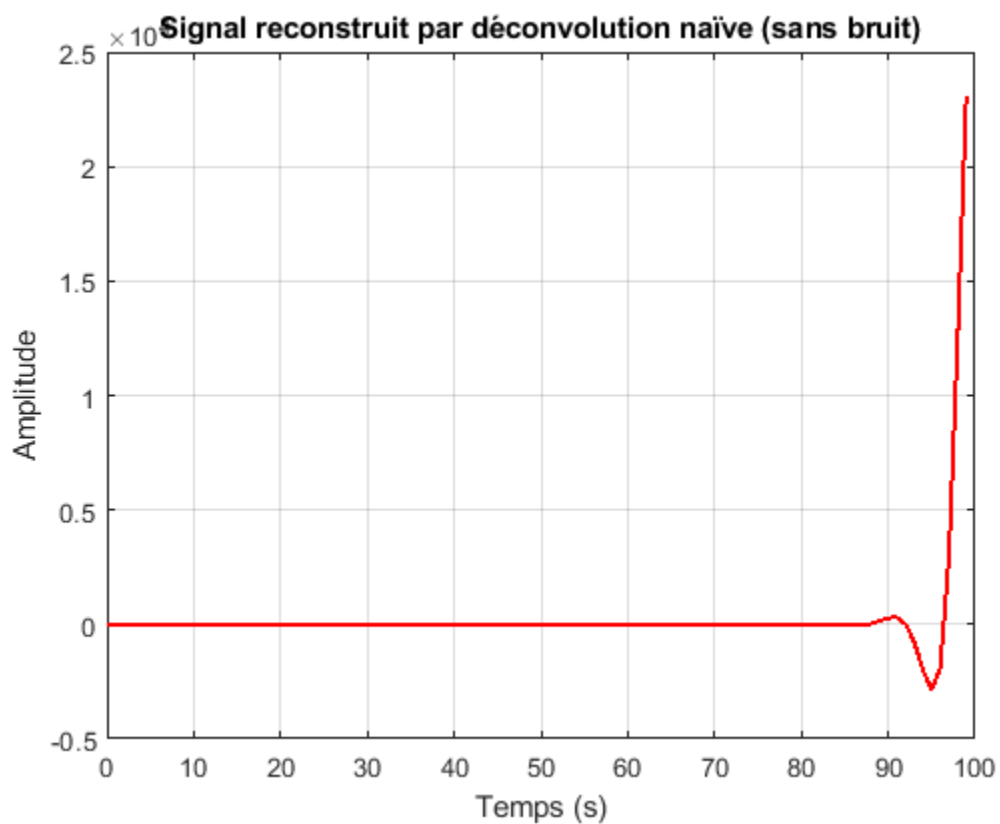
Erreur entre convolution et approche matricielle : 41.5351
```

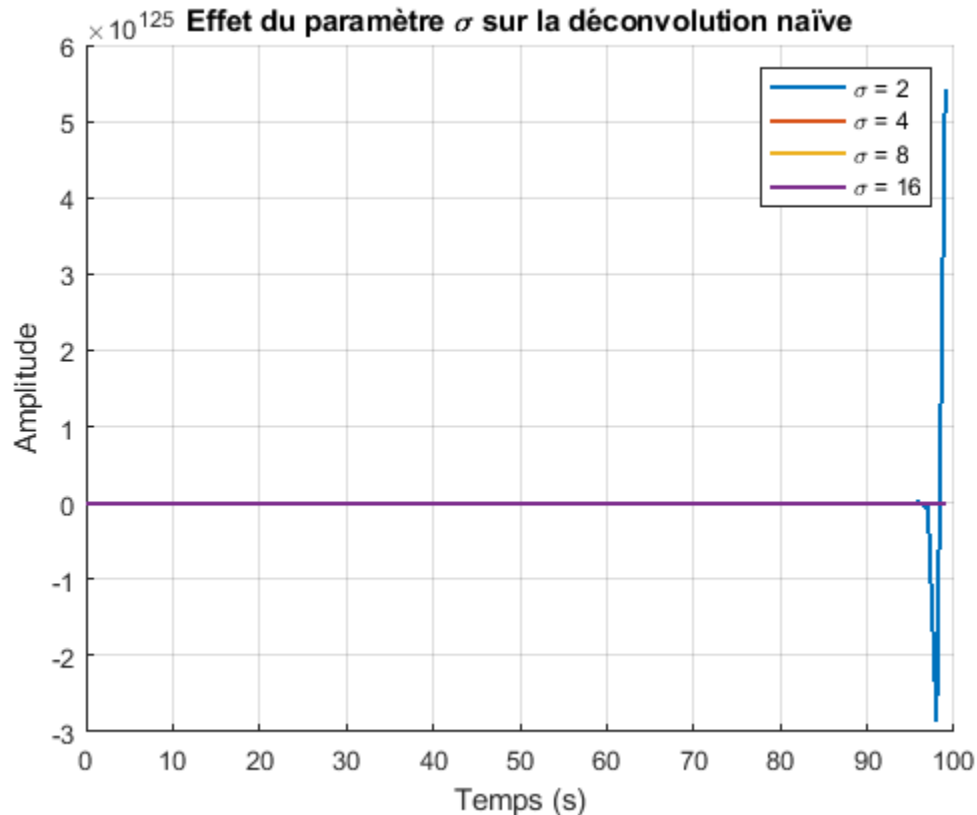




Comparaison entre la convolution et l'approche matricielle sans troncature







2.b Déconvolution par moindres carrés(Ques- tion 14-16)

```
% Calcul du signal déconvolué par moindres carrés
x_ls = pinv(H) * y(:); % Inversion pseudo-inverse

% Affichage du signal reconstruit
figure;
plot(tx, x_ls, 'g', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Déconvolution par moindres carrés');
grid on;
% Interprétation : La méthode des moindres carrés stabilise la solution.

% Calcul de l'erreur de reconstruction
error_ls = norm(x - x_ls'); % Norme de la différence entre le signal
original et reconstruit
disp(['Erreur de reconstruction par moindres carrés : ', num2str(error_ls)]);

% Décomposition en valeurs singulières (SVD)
singular_values = svd(H);
condition_number = singular_values(1) / singular_values(end);
```

```

% Affichage des valeurs singulières
figure;
semilogy(singular_values, 'o-', 'LineWidth', 1.5);
xlabel('Indice');
ylabel('Valeur singulière');
title('Valeurs singulières de H');
grid on;

disp(['Conditionnement de H : ', num2str(condition_number)]);

% question 17:Analyse du conditionnement en fonction de sigma
% Valeurs de sigma à tester
sigma_values = [1, 2, 4, 8, 16, 32];
condition_numbers = zeros(size(sigma_values));

% Parcourir les différentes valeurs de sigma
for i = 1:length(sigma_values)
    sigma_test = sigma_values(i);

    % Construction du filtre gaussien pour sigma donné
    h_test = (1 / (sigma_test * sqrt(2 * pi))) * exp(-((th - m).^2) / (2 *
sigma_test^2));

    % Construction de la matrice Toeplitz associée
    colonne_H_test = [h_test(:); zeros(Nx - 1, 1)];
    ligne_H_test = [h_test(1), zeros(1, Nx - 1)];
    H_test = toeplitz(colonne_H_test, ligne_H_test);

    % Décomposition en valeurs singulières
    singular_values_test = svd(H_test);

    % Calcul du conditionnement (éviter la division par zéro)
    condition_numbers(i) = singular_values_test(1) /
max(singular_values_test(end), eps);
end

% Affichage du conditionnement en fonction de sigma
figure;
semilogx(sigma_values, condition_numbers, 'o-', 'LineWidth', 1.5);
xlabel('\sigma du filtre gaussien');
ylabel('Conditionnement \kappa(H)');
title('Évolution du conditionnement de H en fonction de \sigma');
grid on;

% Affichage du meilleur et pire conditionnement
disp(['Meilleur conditionnement (plus stable) pour sigma = ',
num2str(sigma_values(find(condition_numbers == min(condition_numbers), 1))),
...
', avec \kappa(H) = ', num2str(min(condition_numbers))]);
disp(['Pire conditionnement (plus instable) pour sigma = ',
num2str(sigma_values(find(condition_numbers == max(condition_numbers), 1))),
...
', avec \kappa(H) = ', num2str(max(condition_numbers))]);

```

% c. Déconvolution par régularisation linéaire quadratique (Question 18-20)

% Construction de la matrice D1 (différences premières)

```
dcoll = zeros(Nx,1);  
dcoll(1:2) = [1 -1];  
D1 = toeplitz(dcoll, [1, zeros(1, Nx-1)]);
```

% Tester plusieurs valeurs de lambda

```
lambda_values = logspace(-7, 2, 50);  
errors = zeros(size(lambda_values));
```

```
for i = 1:length(lambda_values)  
    lambda = lambda_values(i);  
    x_reg = (H' * H + lambda * D1' * D1) \ (H' * y(:));  
    errors(i) = norm(x - x_reg);  
end
```

```
figure;  
loglog(lambda_values, errors, 'r', 'LineWidth', 1.5);  
xlabel('\lambda');  
ylabel('Erreur de reconstruction');  
title('Évolution de l'erreur de reconstruction en fonction de \lambda');  
grid on;
```

% Trouver la meilleure valeur de lambda

```
[~, idx] = min(errors);  
lambda_opt = lambda_values(idx);
```

% Calcul de la solution régularisée

```
x_reg_opt = (H' * H + lambda_opt * D1' * D1) \ (H' * y(:));
```

% Affichage du signal reconstruit

```
figure;  
plot(tx, x_reg_opt, 'm', 'LineWidth', 1.5);  
xlabel('Temps (s)');  
ylabel('Amplitude');  
title('Déconvolution régularisée avec \lambda optimal');  
grid on;
```

% Interprétation : La régularisation réduit l'amplification du bruit et améliore la reconstruction du signal.

```
disp(['Valeur optimale de \lambda : ', num2str(lambda_opt)]);
```

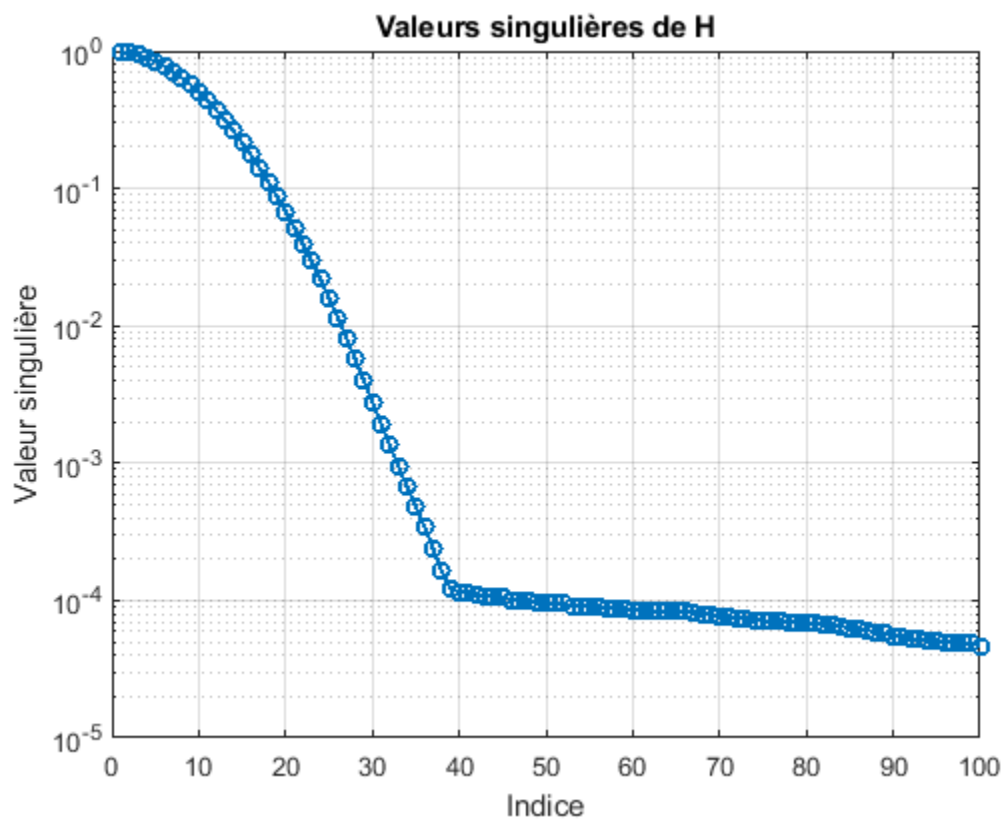
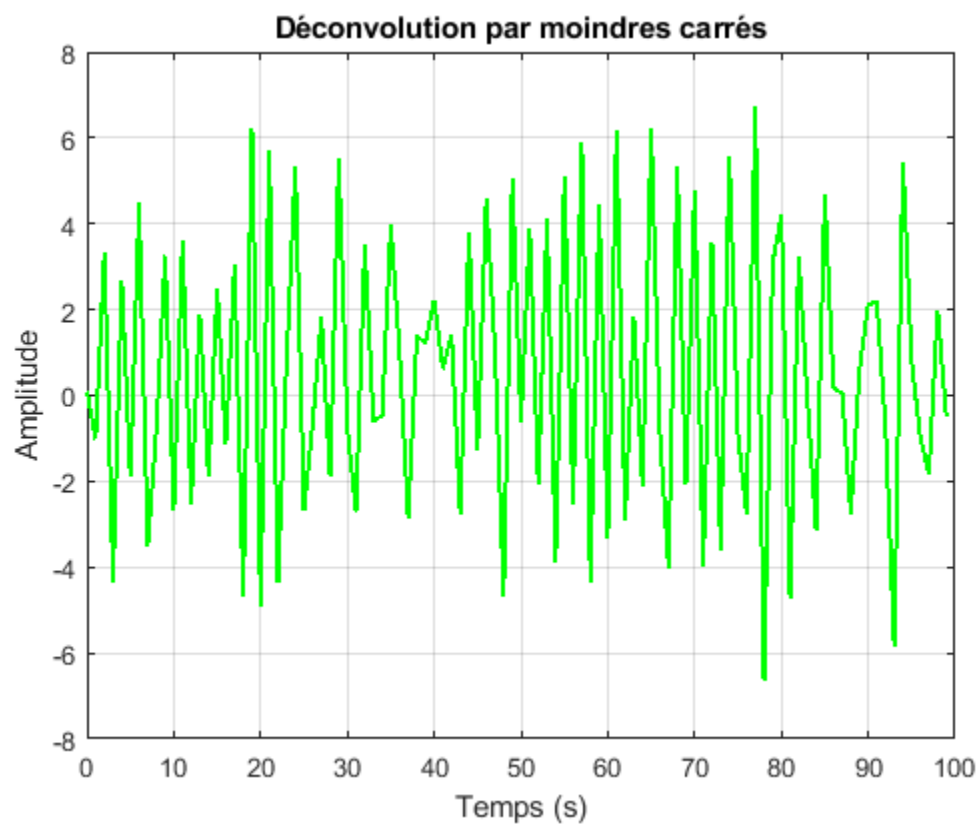
Erreur de reconstruction par moindres carrés : 33.9559

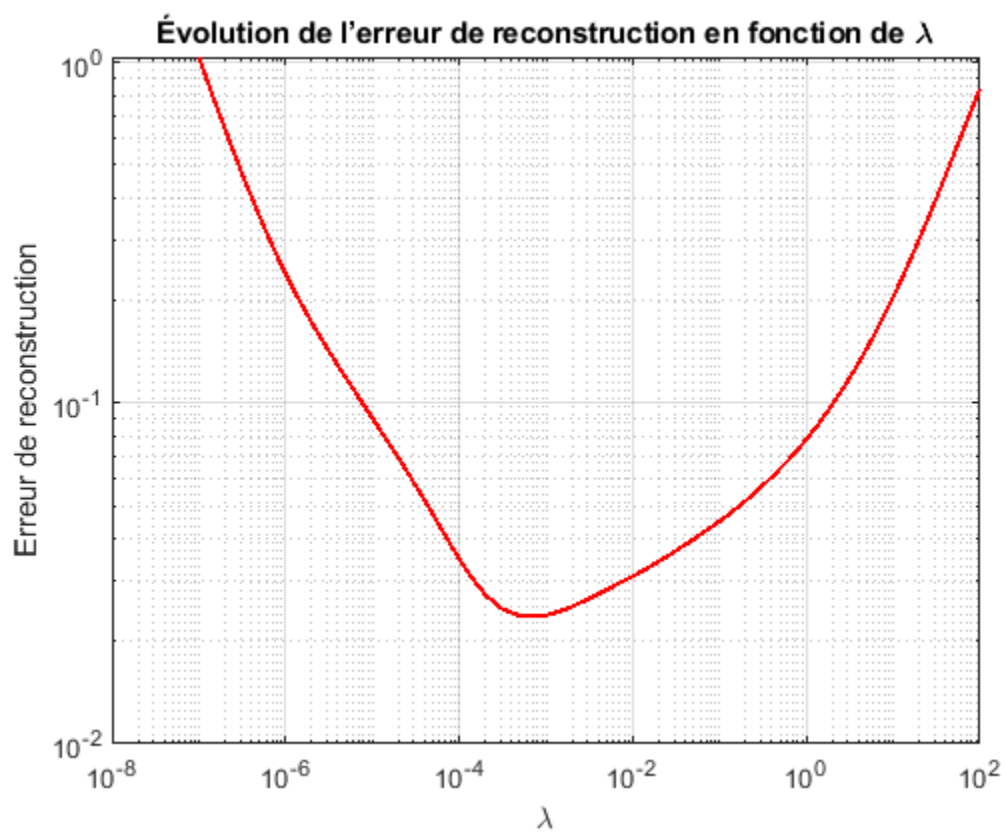
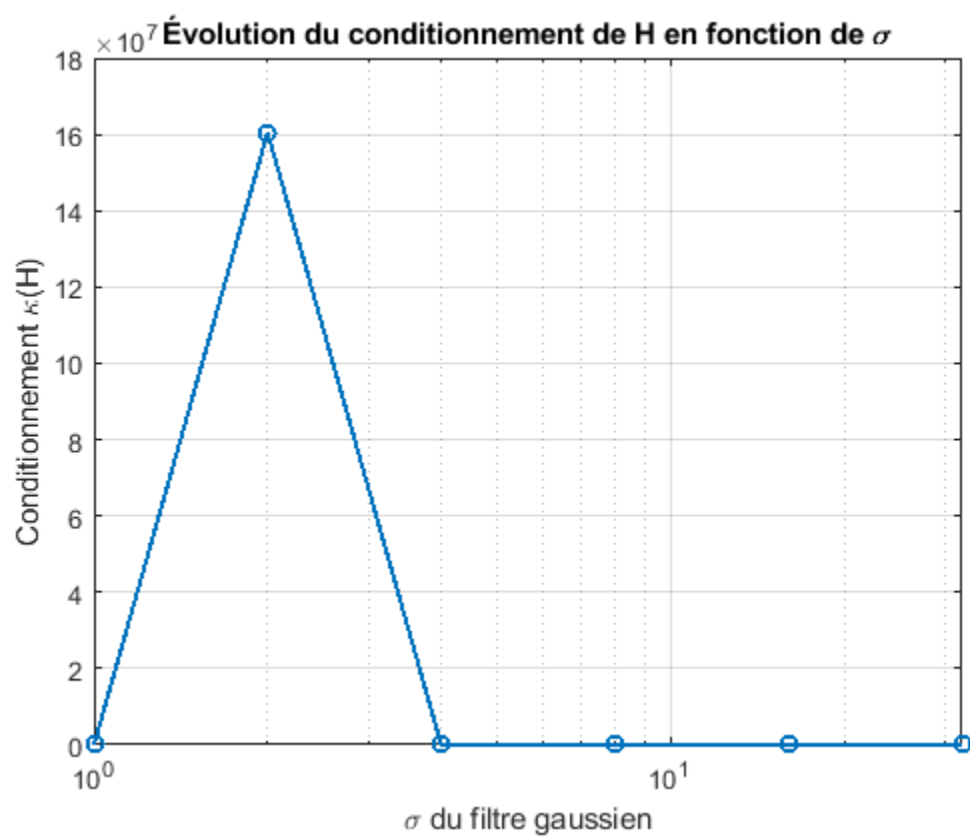
Conditionnement de H : 21225.2935

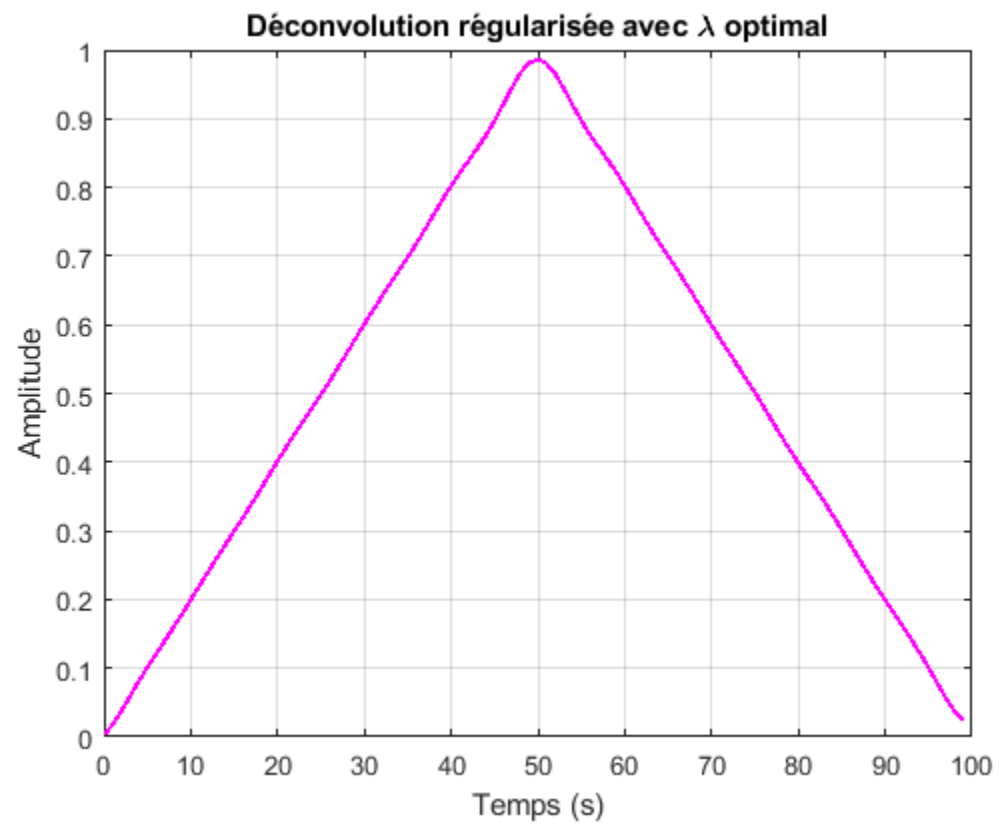
Meilleur conditionnement (plus stable) pour sigma = 1, avec \kappa(H) = 69.1713

Pire conditionnement (plus instable) pour sigma = 2, avec \kappa(H) = 160459034.7682

Valeur optimale de \lambda : 0.00071969







Published with MATLAB® R2024a